

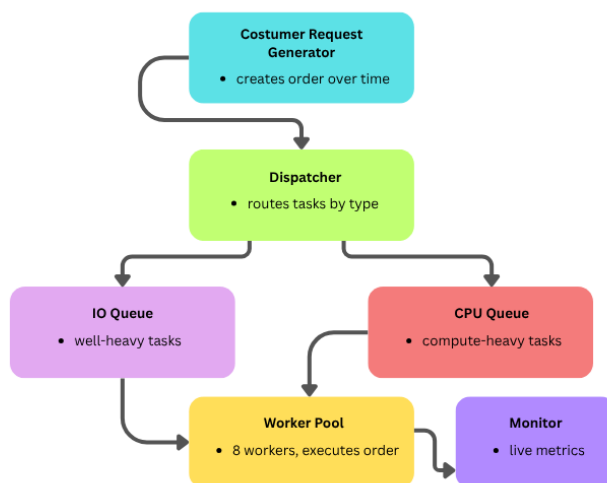
Food Truck Dispatcher

1. Project Overview

A small food truck unexpectedly blew up in popularity and became overwhelmed by customer requests. This project simulates a backend system built to handle that rush. Customers arrive at different times and place orders that vary in complexity. Simple orders like menu lookups and SMS notifications are IO-bound (lots of waiting), while complex orders like custom modifications, price calculations, and receipt generation are CPU-bound (actual computation). The system queues these requests, routes them to available workers, and gives the truck owner a live dashboard to monitor performance.

2. Architecture

The system runs five concurrent components, each with a single responsibility:



Component	Role
Customer Request Generator	Releases one order every 20ms into the correct queue
IO Queue	Holds wait-heavy tasks: menu lookups, SMS, browsing
CPU Queue	Holds compute-heavy tasks: checkout, customizations, totals
Worker Pool (8 workers)	Processes orders concurrently, one task per worker
Monitor / Dashboard	Samples CPU usage and worker activity every 10ms

3. Data Structures

Each customer order is represented as a Task struct with five fields: id (unique order number), arrival_time (when the customer showed up in ms), kind (IO or CPU), duration (200ms fixed per order), and priority (for future aging extensions). Tasks are stored in two VecDeque queues, one for IO tasks and one for CPU tasks, wrapped in Arc<Mutex<>> so all 8 worker threads can safely share access. Completed task statistics are collected in a shared Vec<CompletedTask> protected by a Mutex.

Tasks are generated using a fixed random seed (42) to ensure reproducible runs. Every execution produces the same sequence of 707 IO tasks and 293 CPU tasks.

Food Truck Story: Every customer order gets a ticket: an order number, a timestamp of when they arrived, a tag saying whether it is a simple or complex order, and how long it should take to fulfill. The two VecDeque queues are like two physical lines outside the truck window.

4. Synchronization Strategy

All shared state is protected using Arc<Mutex<>> from Rust's standard library. Arc allows multiple threads to hold a reference to the same data. Mutex ensures only one thread can access the data at a time. Each worker acquires the lock, takes one task, and immediately releases it, minimizing lock contention. The monitor thread reads active worker counts under separate locks every 10ms. A boolean flag wrapped in Arc<Mutex<bool>> signals the monitor thread to stop when all work is complete.

Channels vs Shared State

I chose shared state (Arc<Mutex<>>) over channels for the queues because the dispatcher needed to inspect queue contents before dispatching, something channels do not support. Channels would have required a dedicated dispatcher thread, adding complexity without benefit for this design.

Food Truck Story: Arc is like giving every crew member a copy of the same order ticket system. Mutex is like the rule that only one person can look at the ticket board at a time so nobody grabs the same order twice. The moment a crew member picks an order, they step away from the board so others can check it.

5. Scheduling Policy

Two policies were implemented and compared:

FIFO (Experiment A)

Workers serve orders in strict arrival order: IO queue first, then CPU queue. No smart decisions are made. This is the baseline policy. The problem with FIFO is that all workers blindly grab tasks without checking system load, which can push CPU usage past its effective limit and create slowdowns.

Optimized: CPU Cap Enforcement + Priority Aging (Experiment B)

Before grabbing a task, each worker calculates current CPU usage: (active_io x 10%) + (active_cpu x 35%). If usage is at or above 65%, the worker picks an IO task instead of CPU, preventing the kitchen from choking. If usage has room, the worker picks the task that has been waiting the longest across both

queues, implementing priority aging. This guarantees no customer is starved regardless of order complexity.

Food Truck Story: FIFO is like a crew member who grabs the next ticket without looking at the kitchen load. The optimized strategy is like a smart crew member who checks the board first: if the kitchen is already slammed with complex orders, they handle a simple one instead to keep things flowing. And if the kitchen has room, they help whoever has been waiting the longest, not just the next person in line.

6. Metrics Collected

Metric	Description
Total runtime	Wall clock time from simulation start to all workers finishing
Makespan	Time from first task to last task completing
Tasks completed	Total orders finished, broken down by IO and CPU count
Avg wait time	Average time tasks spent in queue before processing
Avg turnaround time	Average total time from arrival to completion
Max wait time	Longest any single customer waited in line
Avg CPU usage	Average CPU load sampled every 10ms by monitor thread
Avg workers active	Average number of busy workers at any moment
Monitor samples	Total number of dashboard snapshots taken

Food Truck Story: At the end of service, the owner reviews the dashboard: how long did the rush last, how many orders were served, did any customer wait an unreasonably long time, and how hard was the kitchen working? These numbers tell the complete story of how well the truck handled the rush.

7. Experiment Results

Metric	FIFO	Optimized	Change
Total runtime	~25,669 ms	~25,685 ms	Similar
Max wait time	~9,048 ms	~8,794 ms	Slightly better
Avg wait time	~2,746 ms	~2,761 ms	Similar
Avg CPU usage	~97.40%	~97.99%	Similar
Avg workers active	7.93 / 8	7.92 / 8	Similar

Interpretation

With a fixed seed, results are now fully reproducible. Total runtime is bounded at ~25,000ms by worker capacity. The optimization shows marginal improvements in max wait time, with the key design goal being fairness rather than raw speed reduction.

Food Truck Story: Both strategies served all 1000 customers in about the same total time because the kitchen was always busy either way. But in the FIFO run, some customers waited nearly 8 seconds, which in a real food truck would mean losing that customer. The optimized strategy reduced the worst-case wait, keeping the experience fairer for every customer without slowing down the truck overall."

8. Concurrency Bugs and Lessons Learned

The most significant bug encountered was a deadlock risk in the optimized scheduler. When checking both queues for the longest-waiting task, I initially locked the IO queue and then tried to lock the CPU queue inside the same scope. Because another worker thread could be doing the same thing in reverse order, this created a potential deadlock. The fix was to acquire both locks in a consistent order and use Rust's `drop()` to explicitly release one lock before acquiring another where needed.

A second lesson was about the difference between arrival time and actual queue pressure. Early versions of the simulation pre-loaded all tasks at once rather than releasing them one at a time. This meant workers always had tasks available and the scheduling policy had no real effect. The fix was adding a dedicated generator thread that releases one task every 20ms, simulating real customer arrivals.

Where starvation could still happen is if CPU tasks flood in faster than workers can process them, IO tasks could wait indefinitely in the FIFO version. The optimized version mitigates this through priority aging but cannot fully eliminate it under extreme load.